APEX Zero Human Computer — Hand-off

1. One-line summary

APEX is a production-grade, multi-tenant SaaS platform that lets a non-technical business owner type plain English commands ("act as my marketing team") and get an autonomous team of AI agents that builds and runs their company.

---

2. Goal / End state

What "done" looks like:
A 70-year-old or a child can sign up, type "act as my marketing team," and within seconds have a CMO, content writer, SEO specialist, ad manager, and social media manager working autonomously on their business — visible in a live activity feed, self-healing on errors, budget-protected, and paid via Stripe.

Who/what it's for:
Primary: Shane (CEO of A Perfect Mover and Affordable Movers LLC, Everett/Marysville WA) for his moving companies as the first vertical deployment. Secondary: any small business owner who wants to run a company with AI agents instead of human employees. Open source and white-labelable for any vertical.

Why it exists:
Existing multi-agent platforms (CrewAI, AutoGen, LangGraph) are developer tools requiring Python, YAML, or notebook setups. There is no consumer product where the AI builds the AI team. APEX is the first platform where typing English deploys a working company.

---

3. Source of truth

Code:
- Local: `/sessions/gracious-keen-meitner/mnt/apex-zero-human/` (in this Cowork session)
- Windows: `C:\Users\shane\OneDrive\projects\apex-zero-human`
- Production VPS: Hostinger VPS at `/root/apex-zero-human/` (managed via hPanel web terminal)

Database:
- Supabase project: `twsgkmzsayyryqxzfryd` (apex-code)
- Live schema differs from migration files — always query live before writing DB code

Docs/specs (in repo root):
- `INSTRUCTIONS.md` — Master brain, prime directives, execution order, tech stack rules
- `PLAN-SELF-BUILDING-APEX.md` — The godmode self-building plan with research synthesis

- `PLAN-WEDNESDAY-LAUNCH.md` — 3-day launch sprint plan
- `.claude/skills/` — 8 APEX-specific skill files (apex-database, apex-orchestrator, apex-agents, apex-skills-engine, apex-frontend, apex-deploy, apex-vertical-moving, apex-see)

Related projects/dependencies:
- A Perfect Mover and Affordable Movers LLC (the first vertical customer)
- SmartMoving CRM (planned integration via skill)
- RingCentral (planned integration via skill)
- WA UTC 15-C tariff compliance (regulatory requirement for moving vertical)

---

4. Tech stack & architecture

Languages, frameworks, runtimes:
- TypeScript strict mode everywhere
- Node.js 20 LTS
- Next.js 14 App Router
- Tailwind CSS + shadcn/ui
- Turborepo monorepo

Key services:
- Database/Auth/Realtime/Storage: Supabase (Postgres + pgvector + RLS)
- LLM: Anthropic Claude API (claude-sonnet-4-6 strategic, claude-haiku-4-5-20251001 routine)
- Email: Resend
- Payments: Stripe (in progress)
- Hosting: Hostinger VPS + PM2 + Nginx
- Testing: Vitest + Playwright

NOT ALLOWED: Vercel, Railway, Render, AWS, Fly.io, Docker

High-level architecture:

The system runs as three separate PM2 processes on a single Hostinger VPS:

1. **apex-web** (Next.js cluster x2) — User-facing dashboard, auth, all API routes. Talks to Supabase via service role for server-side ops, and to Anthropic API directly for the Command Classifier (Haiku call for intent classification). Includes the CommandBar and ActivityFeed UI.

2. **apex-orchestrator** (Node.js fork) — Autonomous engine that ticks every 5 seconds. Each tick: finds active companies → finds idle agents → routes them to issues via the TaskRouter (with Postgres advisory locks) → executes agents through the heartbeat state machine → records token spend → handles errors via the 5-level self-healing recovery hierarchy. Coordinates 7 core modules: TokenGateway, HeartbeatStateMachine, TaskRouter, EventBus (Postgres LISTEN/NOTIFY), AutoScaler, StallDetector, ModelRouter. Plus: Scheduler + Reactor for routines, ApexMemorySystem (AMS) with pgvector semantic search.

3. **apex-see** (Node.js fork) — Self-Evolution Engine. 6 hidden agents (Sentinel, Cartographer, Alchemist, Crucible, Architect, Chronicle) that monitor AI research, propose improvements, test in shadow environment, deploy approved changes. NEVER visible to operators. Separate process so failures are silent.

The data flow for "act as my marketing team":
```
User types in CommandBar
  → POST /api/apex/command (apex-web)
  → Command Classifier (local pattern match or Haiku call)
  → Auto-Provisioner loads templates/marketing-team/template.json
  → Creates 5 agents in Supabase agents table (all role='cmo' etc.)
  → Creates kickoff issue assigned to CMO
  → Activity Feed (5s polling) shows team deployed
  → Next orchestrator tick: CMO claims issue via TaskRouter
  → Heartbeat: IDENTITY_CONFIRMED → MEMORY_LOADED → PLAN_READ → EXECUTING → HANDOFF_COMPLETE
  → CMO creates sub-issues for content writer, SEO specialist, etc.
  → Each agent picks up their work on subsequent ticks
  → Errors caught → 5-level error-recovery → self-heal or notify
```

---

5. Current status

What's built and working (locally, TypeScript compiles clean):
- Command Classifier (`apps/orchestrator/src/core/command-classifier.ts`) — 9 team types, 50+ synonyms, local pattern match + Haiku LLM fallback
- Auto-Provisioner (`apps/orchestrator/src/core/auto-provisioner.ts`) — Deploys teams with budget check, duplicate detection, hierarchy linking
- 9 team templates in `templates/` (marketing-team, sales-team, support-team, operations-team, dev-team, finance-team, hr-team, moving-company, full-corporation)
- Self-Healing Error Recovery (`apps/orchestrator/src/core/error-recovery.ts`) — L1 retry → L2 reassign → L3 CEO escalate → L4 model fallback → L5 human notify
- Budget Enforcement (`apps/orchestrator/src/core/token-gateway.ts`) — Pre-execution check with 90%/95%/100% graduated alerts
- Activity Feed (`apps/web/src/components/ActivityFeed.tsx` + `app/api/apex/activity/route.ts`) — Live agent visibility, polls every 5s
- Smart Onboarding (`apps/web/src/app/api/auth/onboard/route.ts`) — Optional teamType auto-provisions team at signup
- Command Bar UI (`apps/web/src/components/CommandBar.tsx`) — Plain English input with example commands
- 12 agent implementations in `apps/orchestrator/src/agents/` (CEO, Engineer, QA, EvalEngineer, Marketing, UX, Dispatch, Compliance, FleetCoordinator, LeadRecovery, Quote, ReviewRequest)
- Full orchestrator engine (`apps/orchestrator/src/core/engine.ts`) with all 7 core modules
- 11 built-in skills in `skills/` (web-browser, email-reader, crm-connector, calendar-manager, ads-manager, document-generator, firecrawl, review-requester, phone-listener, compliance-checker, asset-tracker)

- Skill sandbox with VM isolation
- Memory system (AMS) with pgvector
- BYOK encrypted key vault
- Model router with fallback chain
- All 9 dashboard pages with real Supabase data (Command Center, Agents, Issues, Inbox, Spend, Skills, Audit, Companies, Settings)
- Auth flow (Google OAuth + email/password)
- SEE — 6 hidden self-evolution agents

What's built but broken / needs deployment:
- ~120 changed files locally are NOT yet on the production VPS
- Auth callback returns 502 on production (currently bypassed) — needs debug
- Live DB has schema drift from migration files (e.g., `issues.priority` is TEXT not INT, agent roles are mixed case in some places)
- Command route had a case-sensitive role bug (`role='CEO'` vs live `role='ceo'`) — fixed locally with `ilike`, not deployed

What's not started:
- Stripe checkout integration (routes scaffolded, products not created)
- E2E Playwright test suite (manual testing only)
- Moving company vertical wiring (SmartMoving sync, RingCentral listener — skills exist but not integrated)
- Custom domain SSL on production
- Sentry / error monitoring
- Landing page polish

Known bugs / blockers:
1. **Auth callback 502** — biggest blocker, currently bypassed
2. **Git index.lock immutable** in sandbox — cannot commit/push from this environment, must deploy via VPS hPanel terminal using sed/tar
3. **Live schema drift** — always query Supabase live before writing DB code, never trust migration files
4. **Stale dashboard on prod** — VPS still running version before useRealtimeTable + DB table name fixes

---

6. Scope of the port to Claude Code

What SHOULD be brought over:
- Entire `apps/` directory (web + orchestrator)
- Entire `skills/` directory (11 built-in skills + sandbox)
- Entire `templates/` directory (9 team templates)
- Entire `packages/` directory (db migrations + shared utilities)
- All `.claude/skills/*.md` skill files (apex-database, apex-orchestrator, apex-agents, apex-skills-engine, apex-frontend, apex-deploy, apex-vertical-moving, apex-see)
- `INSTRUCTIONS.md`, `PLAN-SELF-BUILDING-APEX.md`, `PLAN-WEDNESDAY-LAUNCH.md`, `HANDOFF.md` (this file)
- `ecosystem.config.js` (PM2 config)
- `nginx.conf`
- `turbo.json`, root `package.json`, `tsconfig.json`
- `deploy-vps.sh`

- All migration files in `packages/db/migrations/` (001 through 017)
- Custom scripts in `scripts/` (apex-test-harness, reset-for-test, store-byok-key)

What should NOT be brought over:
- `node_modules/` — reinstall fresh
- `.next/` build artifacts
- `apex-deploy-features.tar.gz` — old deployment artifact
- Any `.env` files containing real secrets — recreate from `.env.example`
- `tsconfig.tsbuildinfo` files
- Test result caches

Rewrites vs. lift-and-shift:
- **Lift-and-shift** everything currently working
- **Do NOT rewrite** the orchestrator engine, command classifier, auto-provisioner, error recovery, or templates — they compile clean and are production-ready
- **Do reconsider** the inline duplication in `apps/web/src/app/api/apex/command/route.ts` — the classifier and auto-provisioner logic is duplicated from `apps/orchestrator/src/core/` because Next.js can't import directly from the orchestrator package. Long-term fix: extract shared code to `packages/shared/`. Short-term: leave duplicated, it works.

---

7. Constraints & non-negotiables

Tech stack — locked in (from INSTRUCTIONS.md):
- Frontend: Next.js 14 App Router + TypeScript strict + Tailwind + shadcn/ui (no other frameworks)
- Database: Supabase only (no other databases)
- Orchestrator: Node.js 20 + TypeScript, separate PM2 process
- LLM: Anthropic Claude API only (no OpenAI, no Gemini)
- Email: Resend
- Payments: Stripe
- Deployment: Hostinger VPS + PM2 + Nginx (NEVER Vercel, Railway, Render, AWS, Fly.io, Docker)
- Testing: Vitest + Playwright

Prime directives — never violate:
1. Never write code you cannot immediately verify works
2. Never move to the next module until all tests pass
3. Never expose service role keys to client-side code
4. Never create a Supabase table without immediately enabling RLS
5. Never let an agent mutate external state without writing to audit_log first
6. Never route two agents to the same issue simultaneously (use advisory locks)
7. Never make an LLM call without checking token budget first
8. Never import external skills without running the sandbox scanner
9. Never use polling where Supabase Realtime or LISTEN/NOTIFY works
10. Never skip the heartbeat state machine — it is server-enforced, not optional

Design system — APEX Dark Industrial:
- Background: `#0A0A0A` Surface: `#111111` Border: `#1F1F1F` Text: `#F5F5F5` Muted: `#6B6B6B`
- Accent: `#00FF88` (APEX green) — use sparingly
- Warning: `#FFB800` Danger: `#FF4444` Info: `#3B82F6`
- Fonts: Space Mono (display/numbers), DM Sans (body/UI), JetBrains Mono (code)
- Never use purple gradients, never use Inter font

Conventions:
- All agent roles in DB are lowercase (`ceo`, not `CEO`) — case sensitivity matters
- `issues.priority` is TEXT enum (`'critical'|'high'|'medium'|'low'`), not INT
- All TypeScript imports use `.js` extensions for ESM compatibility
- Agent classes extend `BaseAgent` and live in `apps/orchestrator/src/agents/`
- API routes return `{ data, pagination }` shape via `paginatedResponse()` helper
- Frontend hooks: `useRealtimeTable<T>(table, companyId, initialFetch, apiPath?)` for any list data

Things that must not change:
- The heartbeat state machine (server-enforced, 7 strict states)
- The Postgres advisory locks in TaskRouter (prevents double-claims)
- The audit_log append-only requirement (no DELETE, no UPDATE)
- The SEE invisibility (operators must never see SEE tables or APIs)
- The skill sandbox isolation (no direct DB or env access)

---

8. Key files & entry points

Orchestrator (apps/orchestrator/src/):
- `index.ts` — PM2 entry point, boots the engine
- `core/engine.ts` — Main tick loop, processes companies, executes agents, wires error recovery
- `core/command-classifier.ts` — Plain English → structured intent (the brain)
- `core/auto-provisioner.ts` — Deploys teams from templates with guardrails
- `core/error-recovery.ts` — 5-level self-healing hierarchy
- `core/token-gateway.ts` — Budget tracking + pre-execution checks + threshold alerts
- `core/heartbeat.ts` — 7-state machine, server-enforced
- `core/task-router.ts` — Issue claiming with Postgres advisory locks
- `core/event-bus.ts` — Postgres LISTEN/NOTIFY for real-time events
- `core/template-export.ts` — Template import/export with secret scrubbing
- `agents/base-agent.ts` — Base class all agents extend
- `agents/ceo-agent.ts` — CEO agent (the primary command receiver)
- `agents/role-resolver.ts` — Maps role string → agent class
- `models/router.ts` — Smart model router with BYOK and fallback chain

- `memory/ams.ts` — APEX Memory System (pgvector semantic search)
- `routines/scheduler.ts` + `routines/reactor.ts` — Cron + event-triggered routines
- `escalation/stall-detector.ts` — Timeout detection + escalation
- `lib/key-vault.ts` — Encrypted BYOK storage
- `see/*.ts` — Self-Evolution Engine (6 hidden agents)

Web (apps/web/src/):
- `app/api/apex/command/route.ts` — Command Bar API (classifier + auto-provisioner + CEO routing)
- `app/api/apex/activity/route.ts` — Activity feed aggregator
- `app/api/auth/onboard/route.ts` — Smart onboarding with optional team auto-provision
- `app/api/auth/signup/route.ts` — User signup
- `app/auth/callback/route.ts` — OAuth callback (currently 502 on prod)
- `app/(dashboard)/dashboard/page.tsx` — Command Center
- `app/(dashboard)/agents/page.tsx`, `issues/page.tsx`, `inbox/page.tsx`, `spend/page.tsx`, `skills/page.tsx`, `audit/page.tsx`, `companies/page.tsx`, `marketplace/page.tsx`
- `components/CommandBar.tsx` — Plain English command input
- `components/ActivityFeed.tsx` — Live agent activity stream
- `components/RightPanel.tsx` — Sidebar containing CommandBar + ActivityFeed + Inbox
- `lib/hooks.ts` — `useRealtimeTable`, `useActiveCompany`, etc.
- `lib/supabase-server.ts` — Server-side Supabase client + auth helpers

Templates (templates/):
- `marketing-team/template.json` — 5 agents (CMO, Content Writer, SEO, Ad Manager, Social)
- `sales-team/template.json` — 4 agents (Sales Director, SDR, AE, Sales Ops)
- `support-team/template.json` — 4 agents (Support Lead, Tier 1, Tier 2, KB Manager)
- `operations-team/template.json` — 3 agents (Ops Director, Process Analyst, Compliance Officer)
- `dev-team/template.json` — 4 agents (Tech Lead, Backend, Frontend, QA)
- `finance-team/template.json` — 3 agents (CFO, Bookkeeper, AR Specialist)
- `hr-team/template.json` — 3 agents (HR Director, Recruiter, Onboarding Specialist)
- `moving-company/template.json` — Moving vertical (existing)
- `full-corporation/template.json` — 35 agents, 6 hierarchy levels (existing)

Skills (skills/):
- `web-browser/`, `firecrawl/`, `crm-connector/`, `email-reader/`, `calendar-manager/`, `ads-manager/`, `document-generator/`, `review-requester/`, `phone-listener/`, `compliance-checker/`, `asset-tracker/`

Database (packages/db/migrations/):
- `001_foundation.sql` through `017_fix_claim_issue.sql` — Run in numerical order
- 012 is `see_internal` (hidden, deny-all RLS)

- 013-017 are recent fixes (BYOK, heartbeat states, auth callback, claim issue)

Infrastructure:
- `ecosystem.config.js` — PM2 config (apex-web cluster x2, apex-orchestrator fork, apex-see fork)
- `nginx.conf` — Reverse proxy config
- `deploy-vps.sh` — Deployment script
- `turbo.json` — Turborepo pipeline

Docs:
- `INSTRUCTIONS.md` — The master brain (READ FIRST)
- `PLAN-SELF-BUILDING-APEX.md` — Godmode plan
- `PLAN-WEDNESDAY-LAUNCH.md` — Launch sprint plan
- `.claude/skills/*.md` — 8 APEX skill files

---

9. Environment & secrets

Required env vars (names only):
```
# Supabase
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
SUPABASE_SERVICE_ROLE_KEY

# Anthropic
ANTHROPIC_API_KEY

# Email
RESEND_API_KEY

# Stripe (in progress)
STRIPE_SECRET_KEY
STRIPE_WEBHOOK_SECRET
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY

# Encryption
KEY_VAULT_ENCRYPTION_KEY     # for BYOK encrypted at rest

# Orchestrator
ORCHESTRATOR_TICK_MS         # default 5000
NODE_ENV
LOG_LEVEL

# Optional integrations (skills)
FIRECRAWL_API_KEY
SMARTMOVING_API_KEY          # moving vertical
RINGCENTRAL_CLIENT_ID        # moving vertical
RINGCENTRAL_CLIENT_SECRET    # moving vertical
```

External accounts/services needed:

- Supabase project (already exists: `twsgkmzsayyryqxzfryd`)
- Anthropic API account (with budget for sonnet + haiku usage)
- Resend account (for transactional email)
- Stripe account (for billing — not yet set up)
- Hostinger VPS (already provisioned, hPanel access required)
- Firecrawl account (for web research skills)
- Google Cloud project (for OAuth — already configured)

Local dev setup steps:
1. Clone repo / copy files
2. `npm install` at root (Turborepo will install all workspaces)
3. Copy `.env.example` to `apps/web/.env.local` and `apps/orchestrator/.env`
4. Fill in env vars from password manager / Supabase dashboard
5. Run migrations: `npx supabase db push` or apply via Supabase SQL editor
6. Generate types: `npm run generate-types` (writes to `packages/db/types.ts`)
7. Start dev:
   - Web: `cd apps/web && npm run dev` (port 3000)
   - Orchestrator: `cd apps/orchestrator && npm run dev`
8. Visit `http://localhost:3000` and sign up

Production deploy steps (current process):
1. Tarball changed files locally: `tar czf deploy.tar.gz [files]`
2. Upload tarball via Hostinger hPanel web terminal (NOT SSH from sandbox — git index.lock issue)
3. SSH/web-terminal into VPS, extract: `tar xzf deploy.tar.gz -C /root/apex-zero-human/`
4. `cd /root/apex-zero-human && npm install`
5. `cd apps/web && npm run build`
6. `pm2 restart all` (apex-web, apex-orchestrator, apex-see)
7. Monitor: `pm2 logs`

---

10. Open questions / decisions needed

These are unresolved — Claude Code should ask Shane before implementing:

1. **Auth callback 502** — Is the 502 from a Supabase callback URL mismatch, a cookie domain issue, or middleware? Need to debug live before fixing. Currently bypassed.

2. **Stripe pricing tiers** — Free (how many tokens?), Pro (how much, how many tokens?), Enterprise (custom). Token amounts, prices, and trial period not decided.

3. **Token budget defaults** — Currently 1,000,000 tokens per company on signup. Is this the free tier? Should new signups get less?

4. **Custom domain** — When does APEX get a real domain instead of the Hostinger subdomain? SSL needs setup.

5. **Multi-tenant isolation audit** — RLS is enabled but has not been independently audited. Should this happen pre-launch or post-launch?

6. **SEE activation** — SEE is built but is it currently running on production? Should it be enabled for the launch or kept dormant until base is stable?

7. **Moving vertical priority** — Is wiring SmartMoving + RingCentral a Week 1 priority (first customer is Shane's moving company) or Week 2+?

8. **Inline duplication in command route** — The classifier and auto-provisioner are duplicated between `apps/orchestrator/src/core/` and `apps/web/src/app/api/apex/command/route.ts`. Worth extracting to `packages/shared/` now or later?

9. **Activity feed polling vs. realtime** — Currently polls every 5s. Should we use Supabase Realtime subscriptions instead for instant updates? Adds complexity but feels more alive.

10. **CEO escalation loop** — What if the CEO agent itself fails? Currently L3 escalates to CEO, but if CEO is the failing agent there's no L3. Need a fallback (admin notification?).

---

11. First milestone

Deploy everything currently sitting locally to the production VPS and verify the full godmode flow works end-to-end on production.

Concrete deliverable:
1. Tarball all 120+ changed/new files
2. Deploy to `/root/apex-zero-human/` on Hostinger VPS via hPanel
3. Run `pm2 restart all`
4. Open production URL in a browser
5. Sign up as a new user (this requires fixing the auth 502 first OR documenting the bypass)
6. On the dashboard, type "act as my marketing team" in the Command Bar
7. Verify within 30 seconds:
   - 5 marketing agents appear in the Agents page (CMO, Content Writer, SEO Specialist, Ad Manager, Social Media Manager)
   - A kickoff issue appears assigned to the CMO
   - The Activity Feed shows the team deployment event
   - The CommandBar shows "Marketing Team deployed — 5 agents ready" success message
8. Wait 1-2 minutes and verify the orchestrator picks up the kickoff issue (CMO status changes to `working`, heartbeat entries appear, audit log records the execution)
9. Report back with screenshots and any errors

This proves the godmode system works on production. Everything else (Stripe, polish, edge cases) builds on this foundation.

---

12. Glossary

- **APEX** — The product name. Stands for "Autonomous Platform for EXecution." A zero-human company builder.
- **Zero Human** — The product positioning: a company that runs without human employees, where AI agents do all the work.
- **Godmode** — The internal name for the self-building feature: type English, get a working AI team.
- **Cowork** — The Anthropic desktop app that this development is happening inside (a research preview).
- **Heartbeat** — The 7-state machine every agent must traverse for every issue: IDENTITY_CONFIRMED → MEMORY_LOADED → PLAN_READ → RESEARCH_COMPLETE → ASSIGNMENT_CLAIMED → EXECUTING → HANDOFF_COMPLETE. Server-enforced, no skipping.
- **Issue** — A unit of work assigned to an agent. Has title, description, priority, status, assigned_to, parent_id (for hierarchies). Stored in `issues` table.
- **Routine** — A scheduled (cron) or reactive (event-triggered) task that auto-creates issues. Stored in `routines` table.
- **Skill** — A sandboxed capability an agent can use (web browsing, email reading, CRM access, etc.). Stored in `skills/` directory, registered via the skill registry.
- **Persona** — The system prompt that defines an agent's role, mission, rules, and communication style. Stored in `agents.persona`.
- **Heartbeat checklist** — Per-agent list of checks they must run during each heartbeat state. Stored in `agents.heartbeat_checklist` JSONB.
- **AMS** — APEX Memory System. The pgvector-based semantic memory for agents. 5 memory types: identity, rule, plan, learning, context.
- **SEE** — Self-Evolution Engine. 6 hidden agents (Sentinel, Cartographer, Alchemist, Crucible, Architect, Chronicle) that autonomously improve other agents over time. Never visible to operators.
- **BYOK** — Bring Your Own Key. Customers can provide their own Anthropic API key, encrypted at rest, used for their company's LLM calls instead of the platform key.
- **TokenGateway** — The module that checks token budget before every LLM call and tracks usage after. Pre-deducts to prevent race conditions.
- **TaskRouter** — The module that finds the next available issue for an agent's role and claims it via Postgres advisory locks.
- **Auto-Provisioner** — The module that takes a team template and deploys it into an existing company (creates agents, links hierarchy, sets up routines, creates kickoff issue).
- **Command Classifier** — The module that turns plain English commands into structured intents (BUILD_TEAM, EXECUTE_TASK, CONFIGURE, QUERY, UNKNOWN).
- **Activity Feed** — The live UI panel showing every agent state transition, issue change, and audit event in chronological order.
- **5-Level Recovery** — The error recovery hierarchy: L1 retry → L2 reassign → L3 CEO escalate → L4 model fallback → L5 human notify via inbox.
- **Inbox item** — A human-attention notification for the operator. Types: HIRE_APPROVAL, BUDGET_ALERT, STALL_ALERT, PERSONA_PATCH, IRREVERSIBLE_ACTION, SYSTEM_ALERT.

- **Tenant** — A customer organization. One tenant has one organization, one organization has many companies, one company has many agents.
- **Vertical** — An industry-specific configuration of APEX (e.g., moving company vertical includes SmartMoving sync, RingCentral listener, WA UTC 15-C compliance).
- **A Perfect Mover / Affordable Movers LLC** — Shane's two moving companies in Everett/Marysville WA. The first vertical customer for APEX.
- **WA UTC 15-C** — Washington Utilities and Transportation Commission tariff regulation that moving companies must comply with. Drives some compliance agent logic.
- **SmartMoving** — A moving company CRM that APEX integrates with via skill (planned).
- **RingCentral** — A phone system that APEX listens to via skill for missed-call lead recovery (planned).
- **hPanel** — Hostinger's web-based control panel. Includes a web terminal that is the primary deployment interface for this project (since git operations from the Cowork sandbox are blocked by index.lock issues).